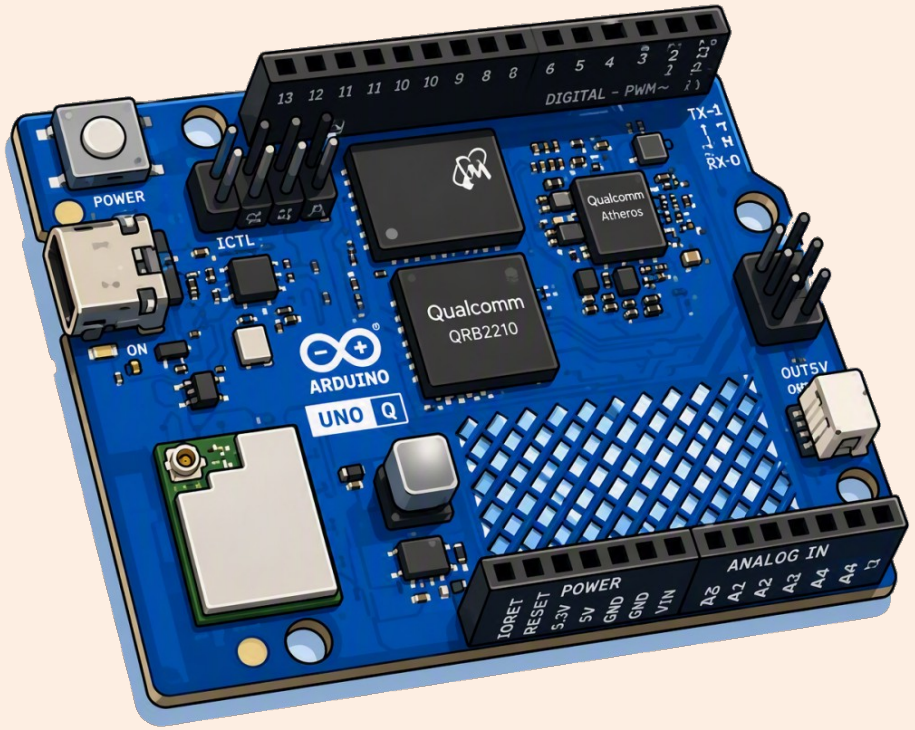
CHAPTER 1 · IESTI05 EDGE AI ENGINEERING

Setting Up the Arduino UNO Q

A terminal-first workflow: ADB, Wi-Fi, SSH

> Debian Linux + Zephyr RTOS
> no GUI · no monitor · no keyboard

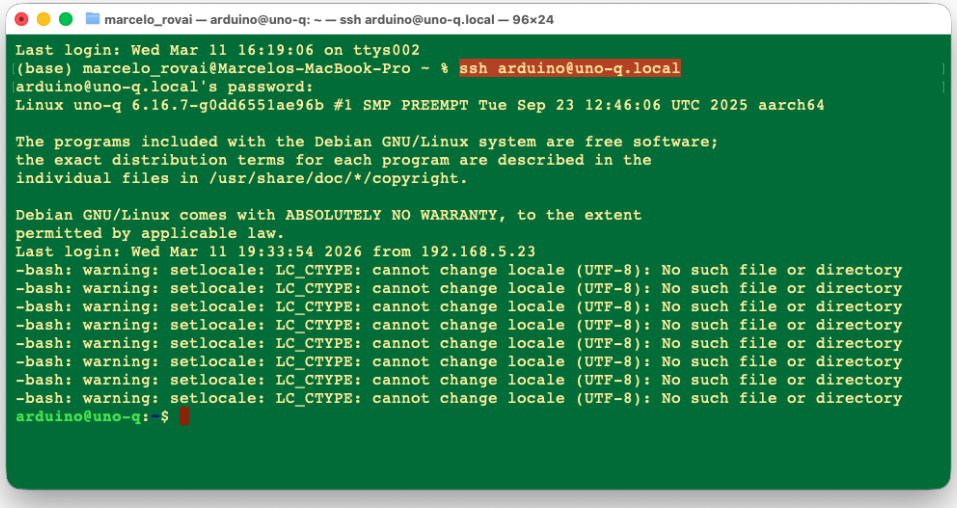
Marcelo Rovai · UNIFEI



WHERE WE ARE GOING

By the end of this chapter you will be here.

An SSH session on the board and a dual-brain app running from one command.



THE ROUTE

Seven stages, one shell at the end

1	**The board**	Two processors, one UNO footprint, an RPC bridge between them.
2	**ADB**	One tool on your computer, talking over the USB-C cable.
3	**Flash**	Optional, but it makes a whole classroom behave the same way.
4	**Connect**	First Linux shell on the board — and change the password.
5	**Wi-Fi + SSH**	Cut the cable. From here the board is a network machine.
6	**Files**	scp for scripts, SFTP for week one, git for anything you iterate on.
7	**Run apps**	arduino-app-cli: one command drives both processors.

STAGE 1 OF 8

What the board actually is

1

Two processors, one UNO footprint, and an RPC bridge between them.

ARCHITECTURE

One board, two processors

MPU · MICROPROCESSOR UNIT

Qualcomm QRB2210

Quad-core Cortex-A53 @ 2.0 GHz

Adreno 702 GPU · dual ISP

Runs Debian Linux — Python, AI inference, networking.

MCU · MICROCONTROLLER UNIT

ST STM32U585

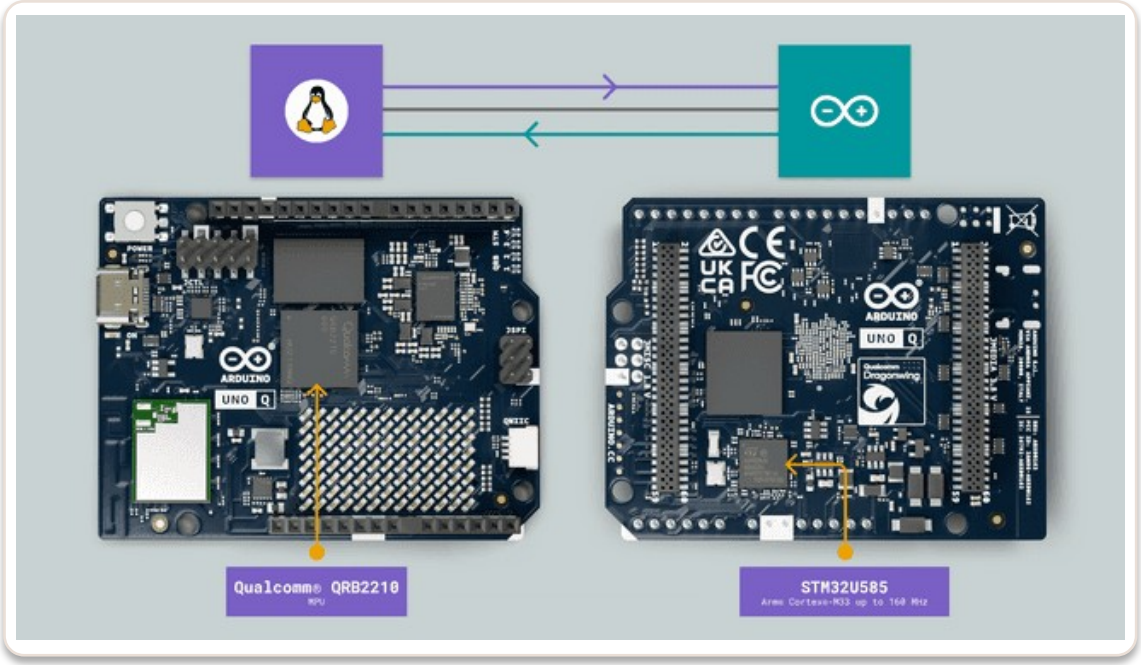
Cortex-M33 @ 160 MHz

Runs the Arduino Core on Zephyr — deterministic, real-time hardware control.

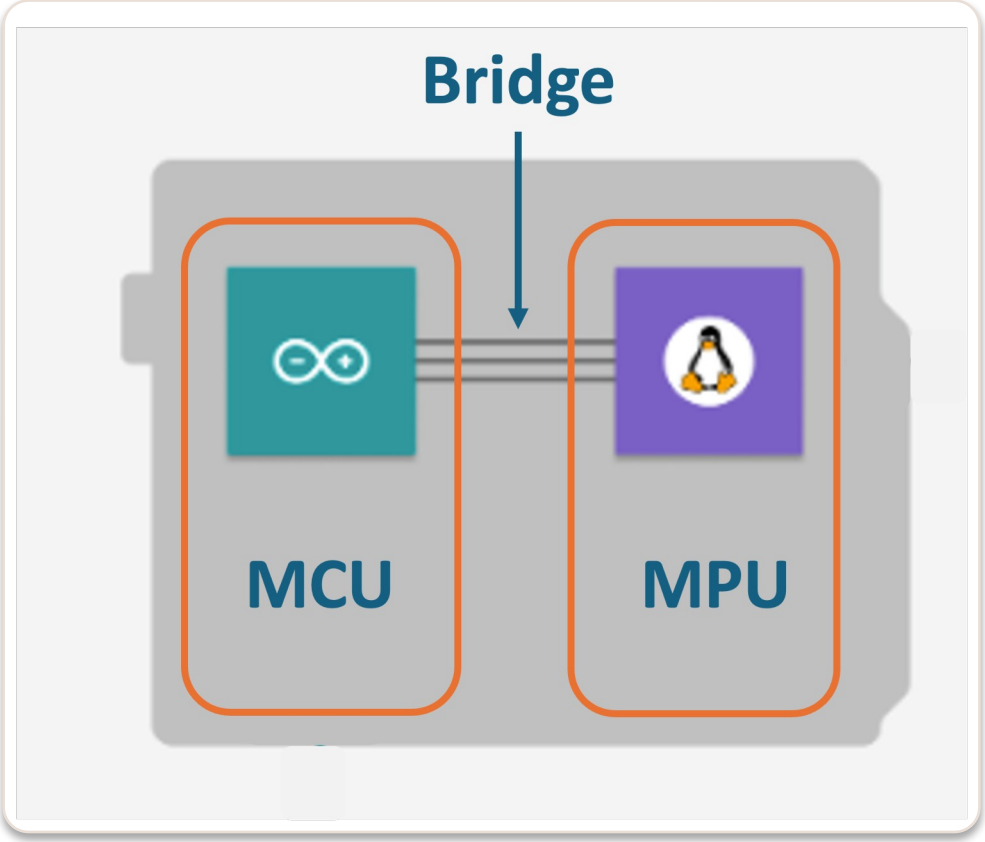
Bridge — Arduino's RPC library — lets Python on the MPU call functions in the sketch on the MCU, and back.

ARCHITECTURE

The bridge between them



UNO Q Architecture



Python on the MPU ⇄ Arduino sketch on the MCU

WHICH ONE TO BUY

Two variants, one architecture

ENTRY

2 GB

LPDDR4X · 16 GB eMMC

Headless SSH development, lightweight edge AI, TinyML.

COMFORTABLE

4 GB

LPDDR4X · 32 GB eMMC

SBC mode with a display, larger AI models, multitasking.

BOTH

Identical

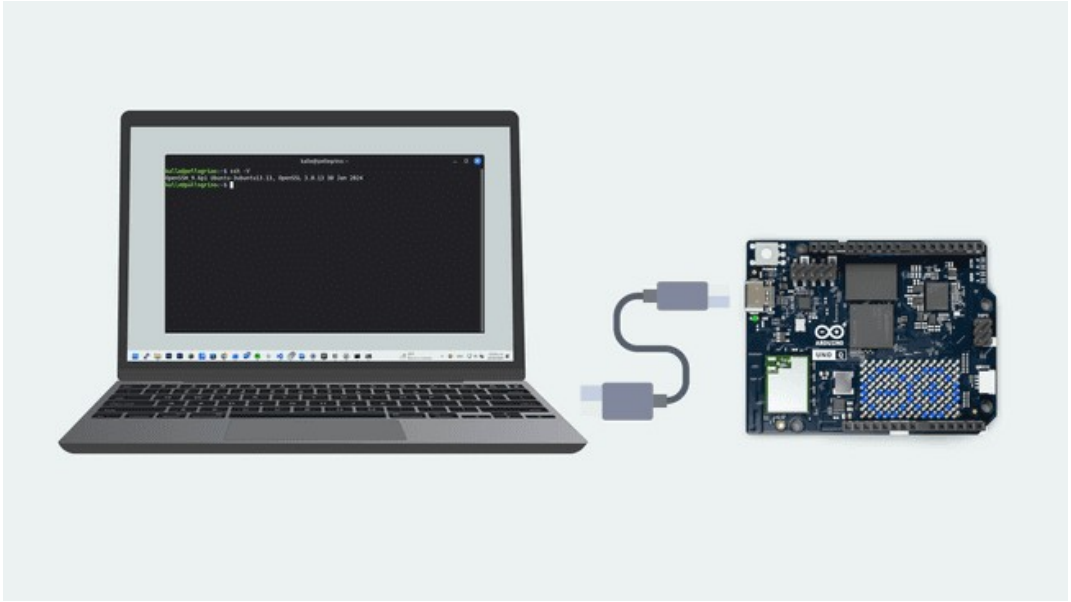
Same processors · Wi-Fi 5 + BT 5.1 · USB-C · UNO headers · Qwiic · 8×13 LED matrix.

RAM is the only difference that matters — and it determines whether generative AI is comfortable.

THE CABLE

The first thing that goes wrong is the cable.

One USB-C port carries power, data, and video. It must be a data cable — not a charge-only one. Some hubs and USB-C adapters are not recognized at all.



Accessing the board shell

APPROACH

Why terminal-first, and not the App Lab GUI

1

Full control

You work inside the board's Debian environment — nothing hidden behind an abstraction.

2

Code lives with you

Projects stay on your machine under Git, not locked inside the board.

3

Transferable skills

Linux, SSH and SCP are what real edge-AI deployments actually use.

4

Honest about failure

You see the actual error, not a spinner. That is where the learning is.

Also: scriptable, and editor-agnostic — VS Code, Vim, anything.

STAGE 2 OF 8

Host setup: ADB

2

One tool on your computer opens a shell on the board over the USB-C cable.

BEFORE YOU START

What you need

HARDWARE

On the bench

Arduino UNO Q (2 GB or 4 GB)

USB-C data cable

Host computer — Linux, macOS or Windows

NETWORK

Same one, both ends

A Wi-Fi network the board and your computer can both reach.

You will need the SSID and password.

HOST SOFTWARE

Three commands

adb — the first headless connection

ssh — remote access over Wi-Fi

scp — file transfer

INSTALL

Getting ADB onto your computer

```
# Linux — Ubuntu / Debian
sudo apt update
sudo apt install android-tools-adb

# macOS — Homebrew
brew install android-platform-tools

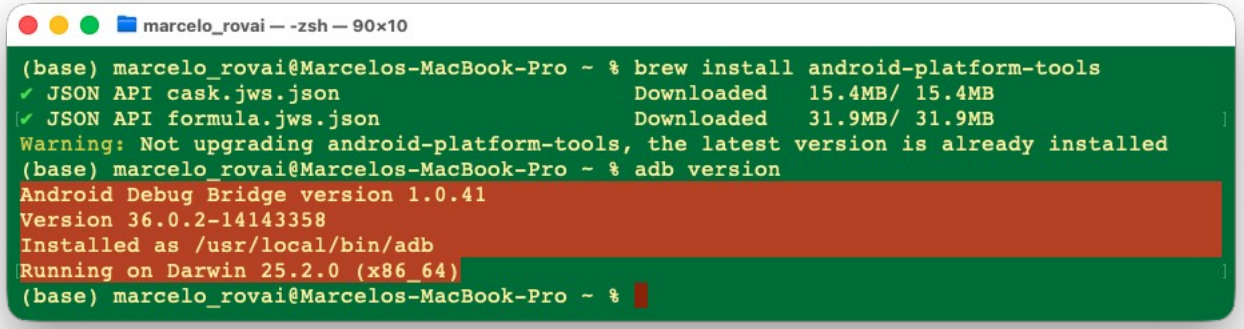
# Windows
Download SDK Platform-Tools, extract to C:\platform-tools,
then add that folder to your PATH
```

VERIFY

Check ADB before you plug anything in

```
$ adb version
Android Debug Bridge version 1.0.41
```

> Run this with the board DISCONNECTED. If it fails here, nothing downstream will work — and you will spend an hour blaming the board.



STAGE 3 OF 8

Flashing the latest image

3

Optional — but essential if you want a whole classroom behaving the same way.

RECOMMENDED FIRST STEP

Why flash a board that already boots

1

Random restarts

The old image has no ZRAM compression — apps restart unpredictably under memory pressure.

2

ADB defaults to root

A security problem, and the wrong habit to learn in the first minute.

3

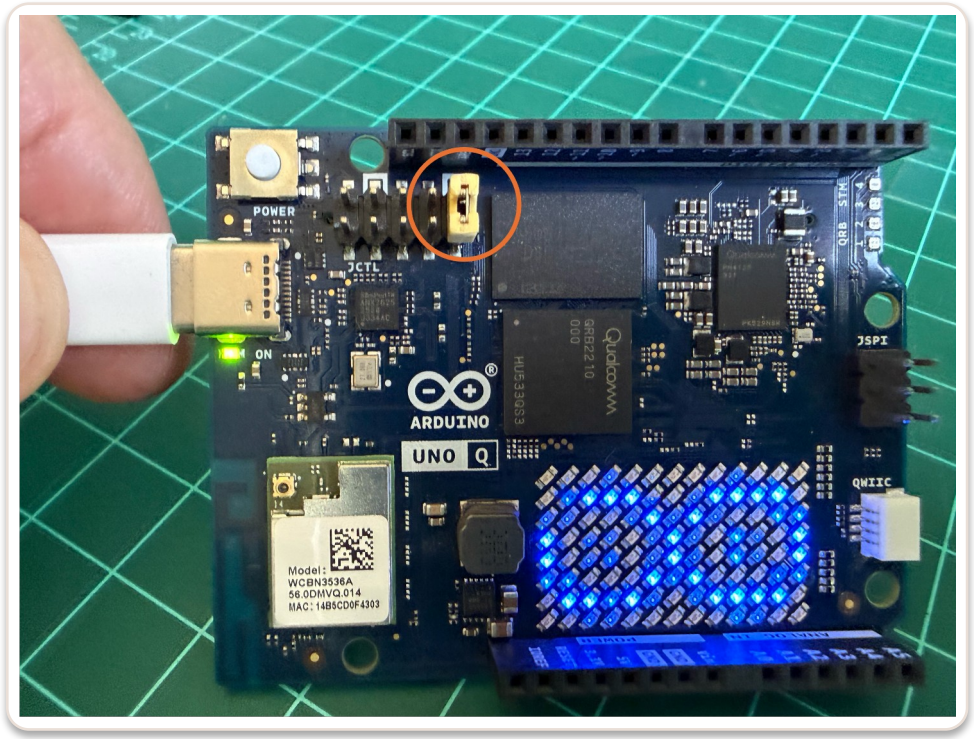
No HDMI audio

Simply missing from the shipped image.

Flashing erases everything — the board goes back to factory state. The tool downloads ~2.3 GB, so use a good connection, or download once and share the image.

STEP 1 OF 3

Jumper the JCTL header — this is EDL mode



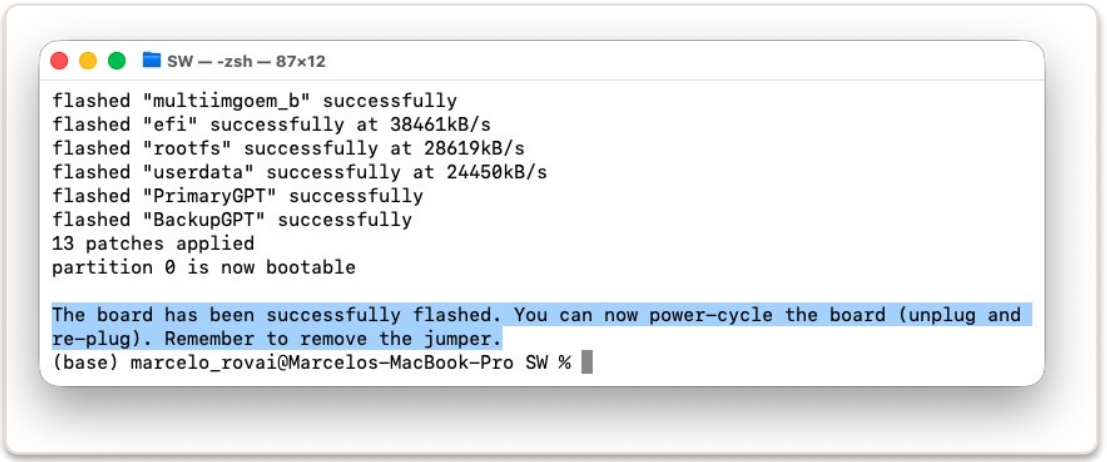
The LED matrix loops the Arduino logo — you are in EDL mode

Board powered OFF. Short the two pins FURTHEST from the USB-C connector, then plug the cable in. Jumper first, cable second.

STEP 2 OF 3

Run the flasher

```
$ ./arduino-flasher-cli flash latest
Download image? [y/N] y
Downloading ~2.3 GB ...
Flashing ...
```



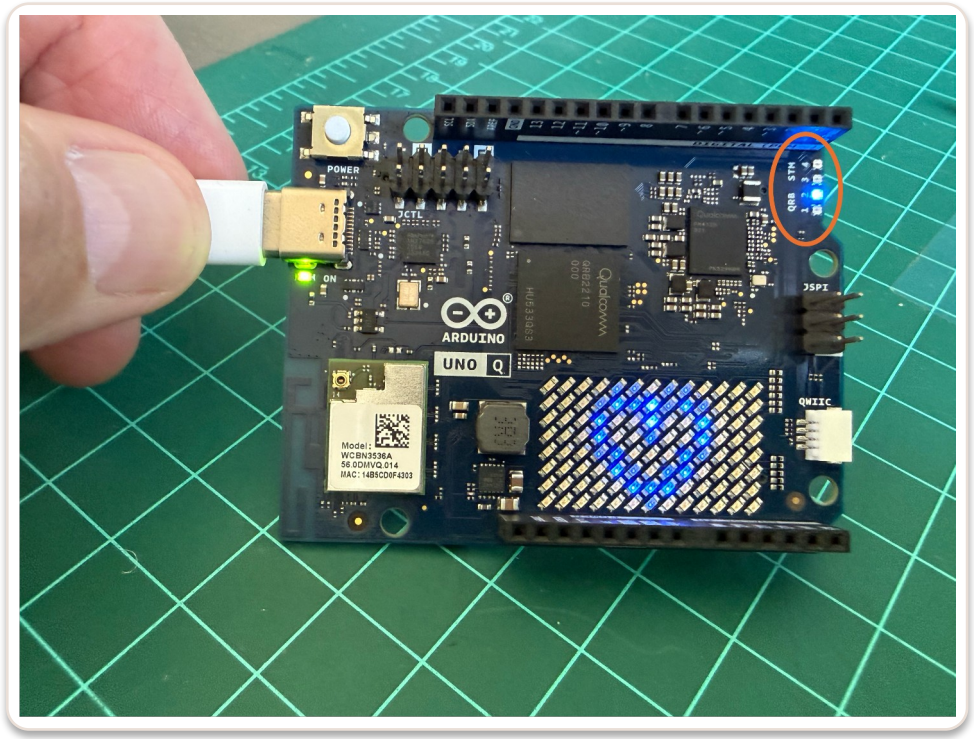
Flashing in progress

> “latest” is deprecated and will be removed; in the future, use “unoq”

> Do not disconnect the cable or interrupt the process. Get arduino-flasher-cli from the Arduino Software Download page or its GitHub releases.

STEP 3 OF 3

Remove the jumper and reboot



Logo animation ends in a heart, blue LED 2 lit — the flash worked

Disconnect · remove the jumper · reconnect.

STAGE 4 OF 8

First connection over ADB

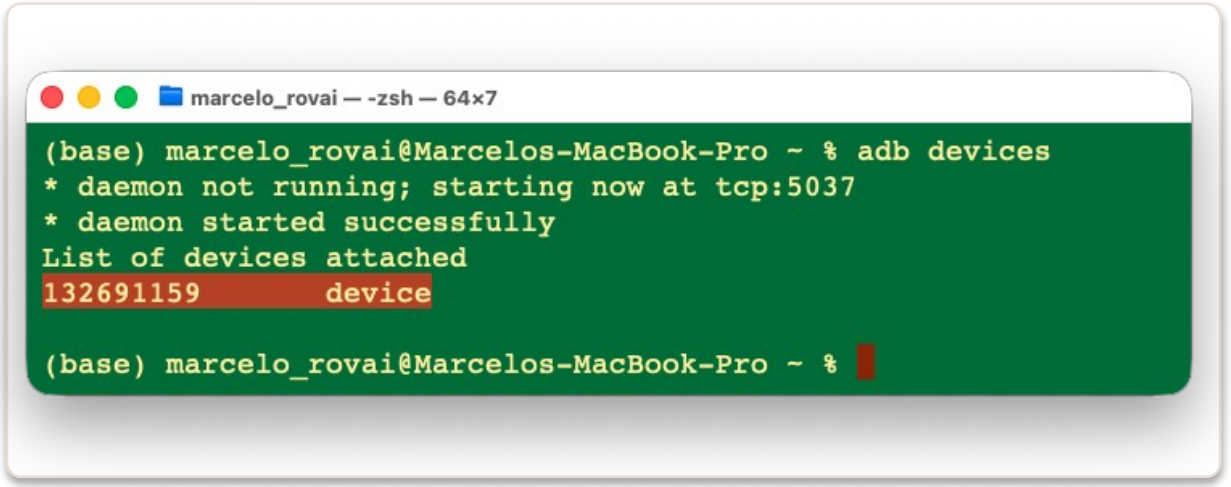
4

Plug in the cable, open a shell, change the password.

HEADLESS BRING-UP

Is the board there?

```
$ adb devices
List of devices attached
XXXXXXXX    device
```



adb devices — the board is seen

> Do not debug anything else until this line shows a device.

IF THE LIST COMES BACK EMPTY

Check these three, in this order

1

The cable

Is it a data cable, or a charge-only one? This is the answer about half the time.

2

The port

Plug directly into the computer. Hubs and USB-C adapters are frequently not recognised.

3

udev rules

On Linux only — without them the device is there but you have no permission to see it.

> Order matters: cable, then port, then udev. Working down the list beats guessing.

YOU ARE ON THE BOARD

Open a shell and change the password

```
$ adb shell
arduino@uno-q:~$

$ sudo passwd arduino
```

Default credentials are arduino / arduino. Change the password NOW — you need the new one for SSH, and the board will keep nagging you until you do.

KNOW WHAT YOU ARE WORKING WITH

Four commands worth running once

```
$ cat /etc/os-release    # which Debian
$ lscpu                  # the four A53 cores
$ free -h                # RAM and swap
$ df -h                  # eMMC space left
```

> Run these on day one. Knowing the real numbers explains a lot of later behavior.

STAGE 5 OF 8

Wi-Fi and SSH

5

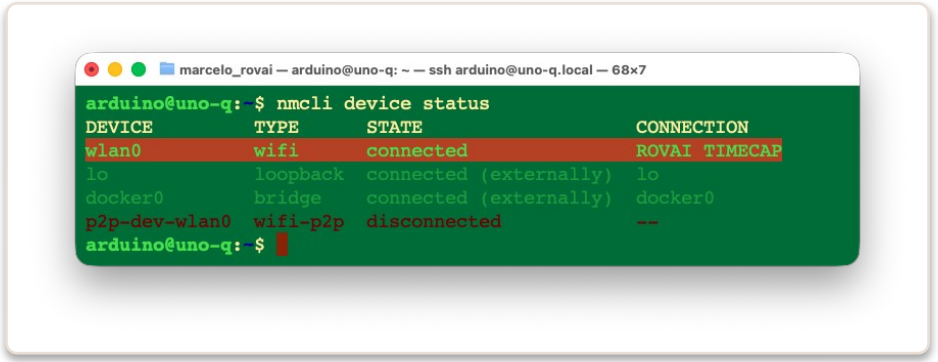
Configure the network from inside the ADB shell, then drop the cable.

NETWORKING

Join a Wi-Fi network with nmcli

```
# inside the ADB shell
$ nmcli dev wifi list

$ sudo nmcli dev wifi connect "YOUR_SSID" \
      password "YOUR_PASSWORD"
```



nmcli device status

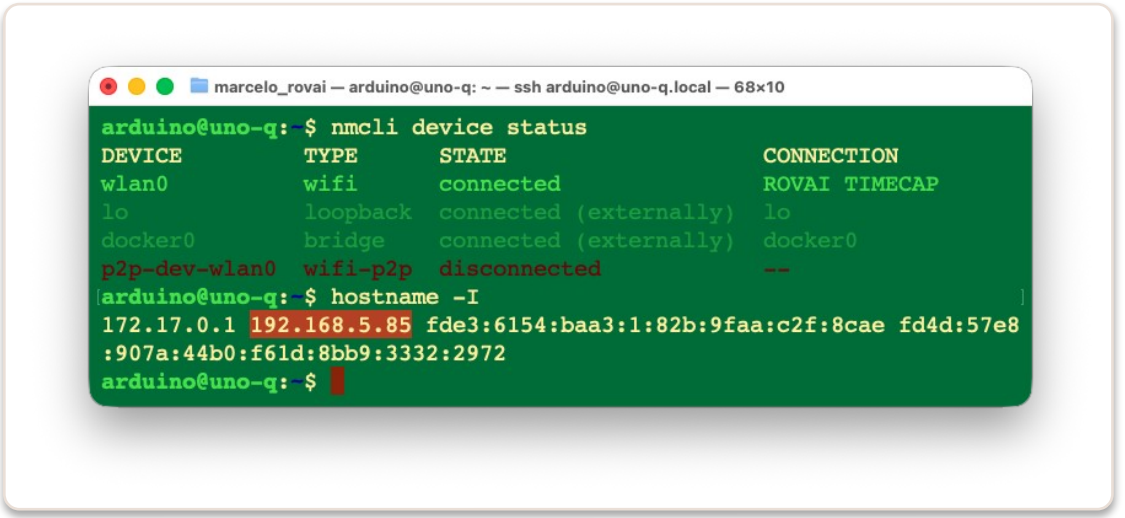
> Quote the SSID if it contains spaces. Some networks hide their 5 GHz SSID — check the band.

NETWORKING

Confirm, and get the address you need

```
$ nmcli device status
wlan0    wifi    connected

$ hostname -I
192.168.5.85
```



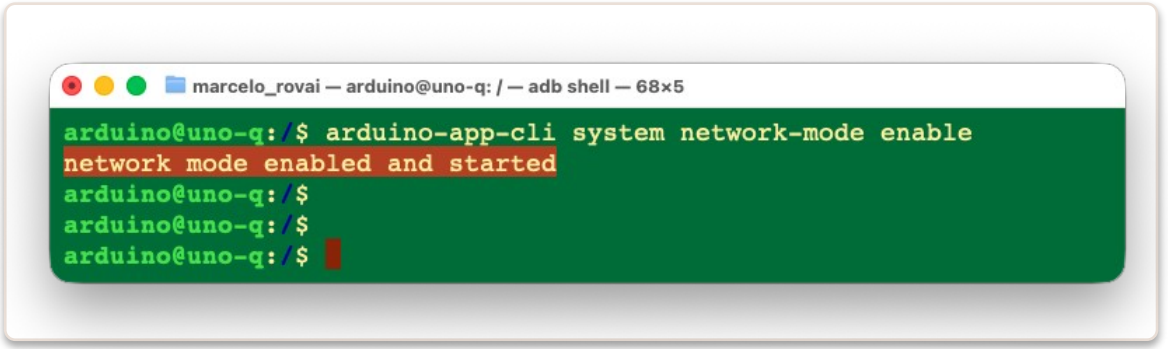
Connected — write this address down

Better than writing it down: take the MAC from ip addr show wlan0 and give the board a static lease on your router, so the address survives reboots.

REMOTE ACCESS

Enable SSH — the step everyone forgets

```
# on the board, in the ADB shell
$ arduino-app-cli system network-mode enable
$ exit
```



network-mode enable

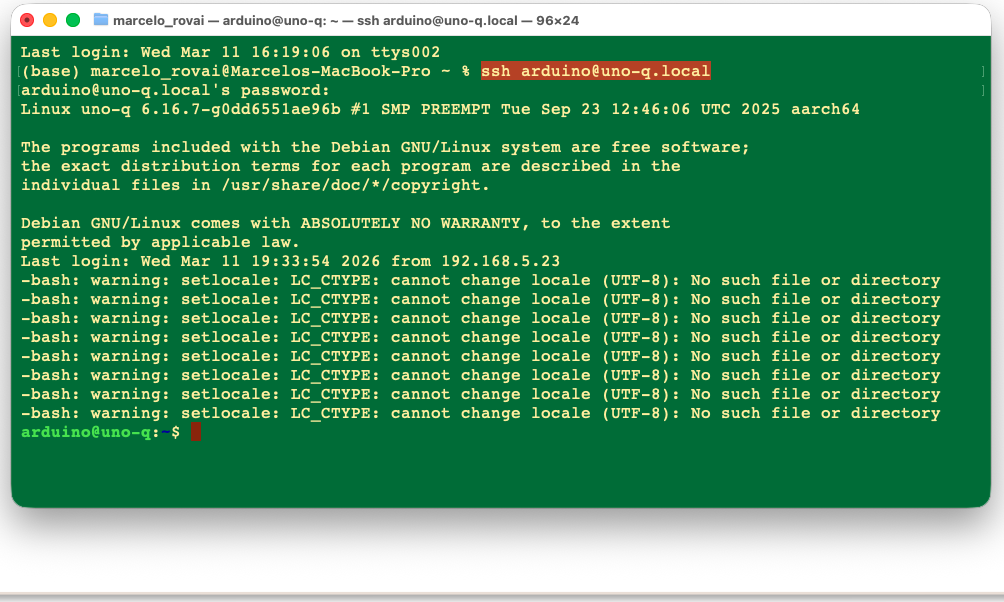
> This is the single most common place people get stuck. SSH exists on the board, but it is not active until this command runs.

REMOTE ACCESS

Drop the cable

```
# from your host computer now
$ ssh arduino@192.168.5.85

# or, on networks with mDNS
$ ssh arduino@uno-q.local
```



Connected over Wi-Fi — the cable is just power now

> Password rejected? Go back in over ADB — adb shell , then sudo passwd arduino — exit, and retry.

HOUSEKEEPING

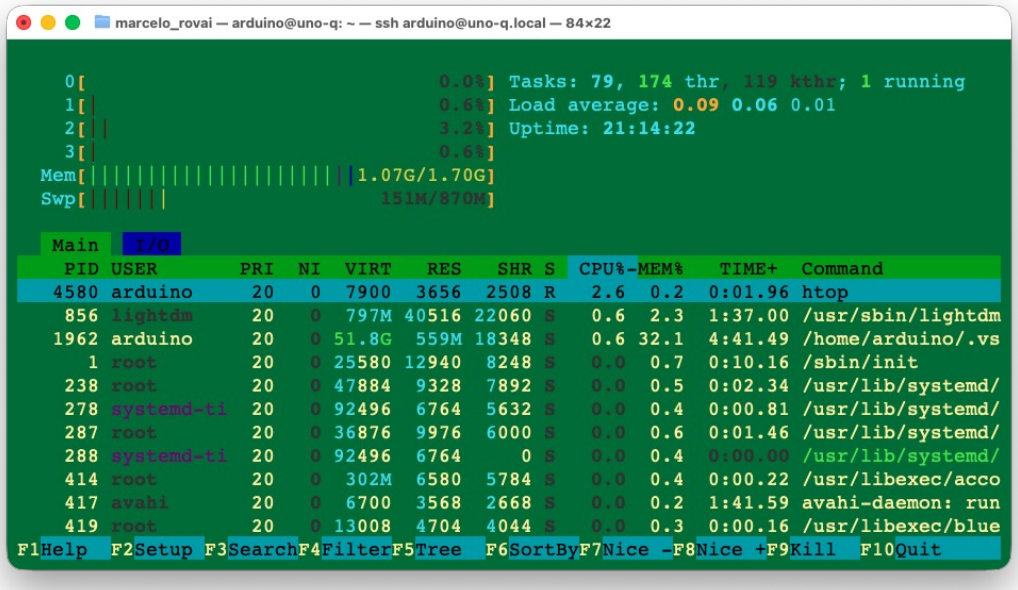
Update the system once

```
$ sudo apt update && sudo apt upgrade
$ sudo reboot
```

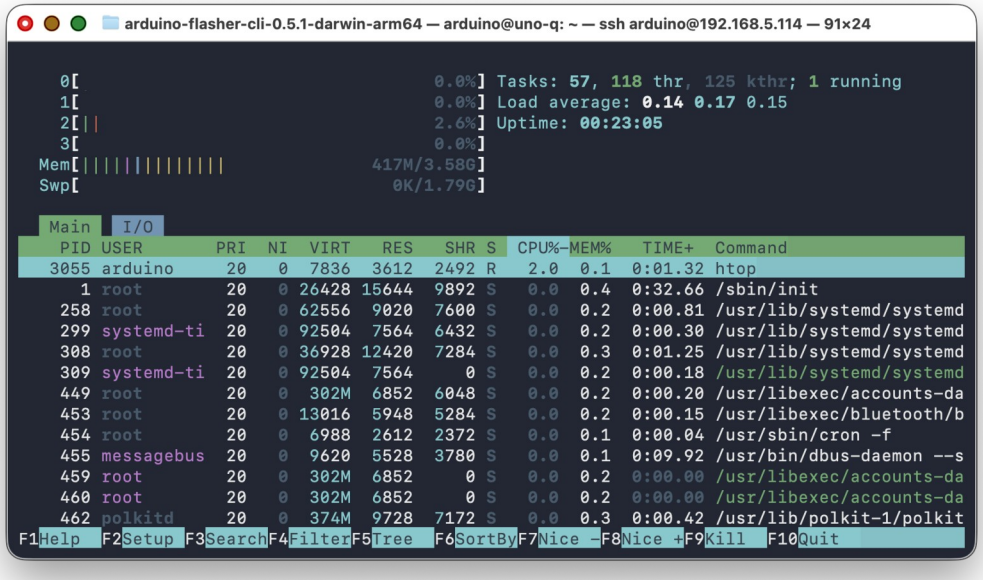
> Do this now, over Wi-Fi, before installing anything else.

KNOW YOUR BOARD

htop: 2 GB versus 4 GB, side by side



2 GB variant



4 GB variant

> Both the RAM and the swap are doubled on the 4 GB board.

STAGE 6 OF 8

Getting code onto the board

6

scp for scripts, SFTP for week one, git for anything you iterate on.

MOVING CODE

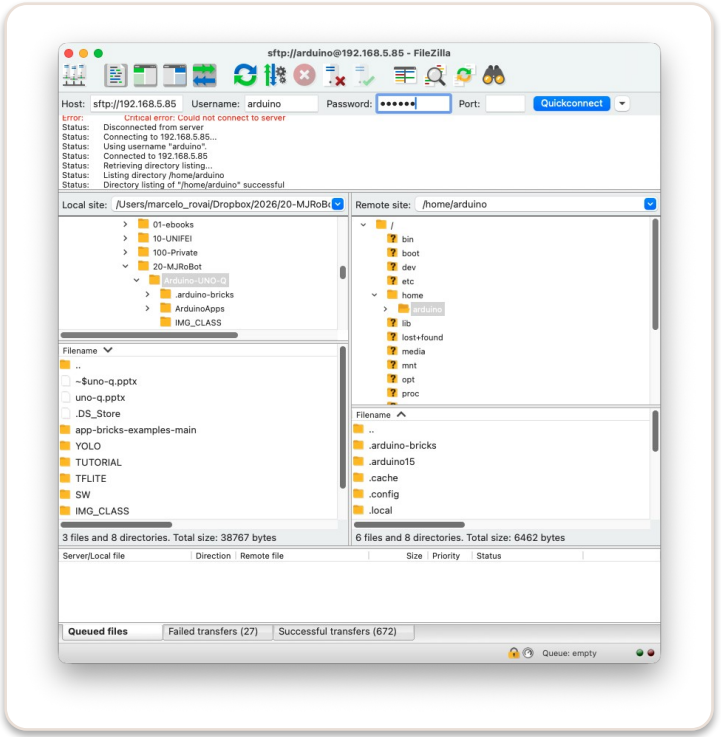
scp — the one that scripts

```
$ cd <your-project-folder>
$ scp -r * arduino@192.168.5.85:~/ArduinoApps/<folder>/
```

> Because it is a plain command, it drops straight into a Makefile or a deploy script. For repeated syncs use rsync — it only sends what changed, which matters over Wi-Fi.

MOVING CODE

SFTP — the one they will actually use



FileZilla: both filesystems side by side

Host field: sftp://192.168.5.85 · board credentials · Quickconnect.

STAGE 7 OF 8

Running things: arduino-app-cli

7

One command orchestrates the Python container and the MCU sketch.

AN IMPORTANT DISTINCTION

Two classes of app

READ-ONLY, CLI-MANAGED

examples:*

Live in /var/lib/arduino-app-cli — not in your home directory, which is why ls never shows them.

arduino-app-cli app start examples:inspirational/blink

YOURS, EDITABLE

user:*

Live in ~/ArduinoApps/. The only place ls shows anything, and the only place you edit.

arduino-app-cli app new "Blink"

This trips everyone up: app list --examples shows examples you cannot find with ls. They are managed by the CLI.

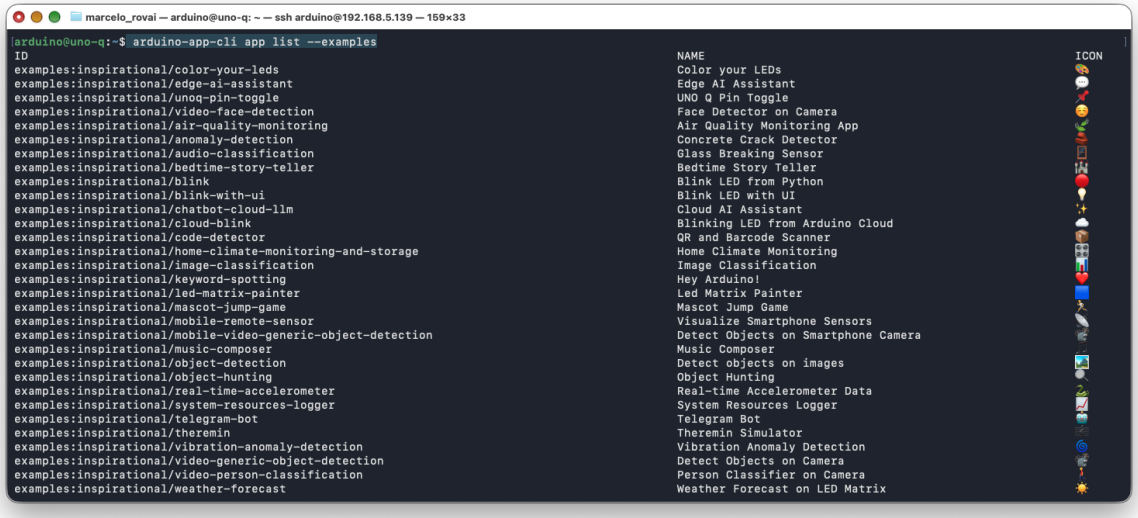
BEFORE YOU EXPLORE

Pull the latest components

```
$ arduino-app-cli system update
```

> This takes a long time. Start it before a break, not in the middle of a demo.

```
$ arduino-app-cli app list --examples
```



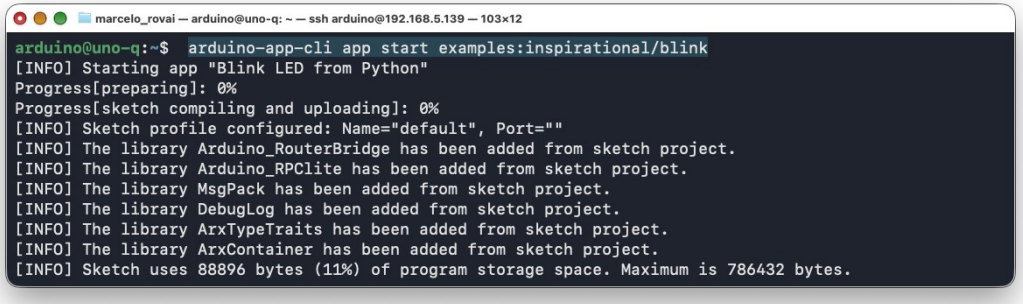
app list --examples — both classes, one listing

THE FIRST DUAL-BRAIN APP

Start it

```
$ arduino-app-cli app start
examples:inspirational/blink
```

The FIRST run takes several minutes — it downloads Arduino libraries and builds the Python container. Warn people, or half of them will decide it is broken and reboot.



Starting the app

When loaded, you will get:

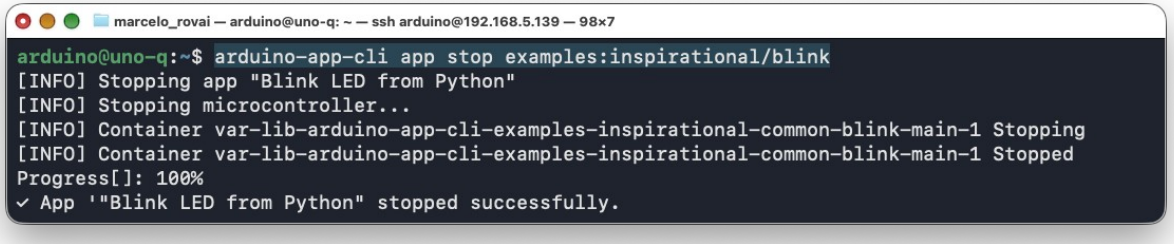
Progress[]: 100%
✓ App "Blink LED from Python" started successfully

THE FIRST DUAL-BRAIN APP

Watch it, then stop it

```
$ arduino-app-cli app logs
examples:inspirational/blink

$ arduino-app-cli app stop
examples:inspirational/blink
```



app logs — the Python side talking

> Add --follow to tail the log live.

REFERENCE

The arduino-app-cli commands you will actually use

Command	What it does
`arduino-app-cli app new "<name>"`	Create a new app skeleton in ~/ArduinoApps/
`arduino-app-cli app start <path>`	Build if needed, then start the app
`arduino-app-cli app logs <path>`	View the Python-side log — add --follow to tail it
`arduino-app-cli app stop <path>`	Stop a running application
`arduino-app-cli app list`	List your apps — add --examples for the examples
`arduino-app-cli app ps`	Show which apps are running right now
`arduino-app-cli system update`	Update the CLI and board components (slow)
`arduino-app-cli system cleanup`	Remove unused containers — reclaim eMMC space

app start compiles the sketch, flashes the MCU, builds the Python environment and launches the app — both processors, one command.

SURVIVAL KIT

The Linux commands this course leans on

SYSTEM

```
cat /etc/os-release
lscpu
free -h
df -h
htop
```

NETWORK

```
ip addr
nmcli device status
nmcli dev wifi list
hostname -I
```

PACKAGES

```
sudo apt update
sudo apt upgrade
sudo apt install <pkg>
sudo apt autoremove
```

FILES

```
ls -lah
cd / mkdir / cp / mv
rm -rf <dir>
cat / nano <file>
```

SERVICES

```
sudo systemctl status ssh
sudo systemctl restart ssh
sudo reboot
sudo shutdown -h now
```

CLEANUP

```
df -h
sudo apt clean
arduino-app-cli system cleanup
docker system prune -a
```

Standard Debian — anything you know from a Raspberry Pi or a cloud VM applies here unchanged.

STAGE 8 OF 8

When it breaks — and what it is for

8

The failures you will meet in week one, then an honest look at the board's limits.

WEEK ONE

The failures you will actually hit

1

ADB does not see the board

Charge-only cable, a hub in the path, or missing udev rules on Linux.

2

SSH connection refused

network-mode enable was never run — or you are on a different network.

3

SSH password rejected

Reset it over ADB: adb shell , then sudo passwd arduino .

4

Everything is mysteriously slow

Check htop. If swap is in use, close something — the A53s do not recover from thrashing.

Nearly all of these are cable, network, or memory. Teach the diagnosis order and they stop being interesting.

BE REALISTIC

What this board is not good at

COMPUTE

Roughly a Pi 3

Cortex-A53 cores. Models beyond ~2B parameters are impractical. For heavy inference a Pi 5 or an accelerator is still the right answer.

MEMORY & PORTS

Tight, and singular

2 GB is tight for editor + container + model. One USB-C port shares power, data and video — SBC mode needs a hub.

SILICON & SOFTWARE

No NPU, pre-1.0 tools

Inference runs on CPU and GPU. App Lab and the CLI are pre-1.0 — expect rough edges. No built-in camera.

State the ceiling on day one: students who know it design within it, instead of blaming themselves.

CARRY THESE OUT OF CHAPTER 1

Three things

1

The board is two computers

An MPU running Debian and an MCU running Zephyr, talking over Bridge. Knowing which side a problem is on is half of debugging it.

2

The terminal is the interface

ADB to bootstrap, Wi-Fi to cut the cord, SSH from then on. Code lives on your machine under Git; the board just runs it.

3

One CLI drives both processors

app new, app start, app logs, app stop. Every chapter from here uses exactly those four.

RESOURCES

Where to read further

Resource	Where
UNO Q documentation	docs.arduino.cc/hardware/uno-q
UNO Q datasheet (PDF)	docs.arduino.cc/resources/datasheets/ABX00162-datasheet.pdf
Arduino App CLI tutorial	docs.arduino.cc/software/app-lab/tutorials/cli
Arduino App CLI (GitHub)	github.com/arduino/arduino-app-cli
Edge Impulse — UNO Q	docs.edgeimpulse.com/hardware/boards/arduino-uno-q
arduino-flasher-cli releases	github.com/arduino/arduino-flasher-cli/releases

Next — Ch. 2, Generative AI at the Edge: building llama.cpp from source, running Qwen3.5 from the CLI and a local llama-server, and calling it from Python.

Questions?

Prof. Marcelo J. Rovai

rovai@unifei.edu.br

UNIFEI — Federal University of Itajubá, Brazil
AiEng4D — Academic Network Co-Chair
EdgeAIP — Academia-Industry Partnership Co-Chair



